

ANÁLISIS ESTÁTICO DE CÓDIGO

Informe de Linter — Entrega 05

Ingeniería de Software 1 (2016701)

Universidad Nacional de Colombia — Sede Bogotá

Proyecto	Bolirana — Plataforma de apuestas deportivas
Herramienta	Checkstyle 9.3 (maven-checkstyle-plugin 3.3.1)
Alcance	Backend Spring Boot — src/main/java/
Reglas	checkstyle.xml personalizado (14 reglas)
Resultado	46 violaciones → 0 violaciones (BUILD SUCCESS)
Fecha	Junio 2025

1. Introducción

El análisis estático de código permite detectar problemas de calidad, estilo y consistencia sin ejecutar el programa. Para el backend de Bolirana se utilizó Checkstyle 9.3, una herramienta ampliamente adoptada en el ecosistema Java que verifica el cumplimiento de convenciones de codificación definidas en un archivo de configuración XML. Su integración con Maven permite ejecutarlo en cualquier momento con un único comando y generar reportes automáticos.

2. Herramienta utilizada y configuración

2.1 Herramienta

Atributo	Detalle
Herramienta	Checkstyle
Versión	9.3 (vía maven-checkstyle-plugin 3.3.1)
Integración	Plugin Maven — <code>./mvnw checkstyle:check</code>
Config. base	Personalizada — <code>checkstyle.xml</code> en la raíz del proyecto
Reporte HTML	<code>target/site/checkstyle.html</code> (<code>./mvnw checkstyle:checkstyle</code>)
Reporte XML	<code>target/checkstyle-result.xml</code>

2.2 Plugin en pom.xml

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-checkstyle-plugin</artifactId>
  <version>3.3.1</version>
  <configuration>
    <configLocation>checkstyle.xml</configLocation>
    <failsOnError>>false</failsOnError>
    <consoleOutput>>true</consoleOutput>
  </configuration>
</plugin>
```

2.3 Reglas configuradas (checkstyle.xml)

Se definió un `checkstyle.xml` personalizado con 14 reglas. La regla `LineLength` fue ubicada al nivel `Checker` (requerimiento de Checkstyle 9.3 — no puede anidarse dentro de `TreeWalker`).

Regla	Módulo padre	Descripción
<code>MissingJavadocMethod</code>	<code>TreeWalker</code>	Javadoc obligatorio en métodos public
<code>TypeName</code>	<code>TreeWalker</code>	Clases e interfaces en PascalCase
<code>MethodName</code>	<code>TreeWalker</code>	Métodos en camelCase
<code>ConstantName</code>	<code>TreeWalker</code>	Constantes en UPPER_SNAKE_CASE
<code>LeftCurly</code>	<code>TreeWalker</code>	Llave { de apertura en la misma línea
<code>RightCurly</code>	<code>TreeWalker</code>	Llave } con else en la misma línea
<code>AvoidStarImport</code>	<code>TreeWalker</code>	Prohíbe imports con wildcard (<code>java.util.*</code>)

Regla	Módulo padre	Descripción
ModifierOrder	TreeWalker	Orden correcto: public static final
LocalVariableName	TreeWalker	Variables locales en camelCase
ParameterName	TreeWalker	Parámetros en camelCase
WhitespaceAfter	TreeWalker	Espacio obligatorio después de comas
Regexp (trailing ws)	TreeWalker	Prohíbe espacios al final de línea
LineLength (max=120)	Checker	Líneas no deben superar 120 caracteres
NewlineAtEndOfFile	Checker	Archivos deben terminar con nueva línea

3. Resultados — ANTES de la corrección

Se ejecutó `./mvnw checkstyle:check` sobre el código original sin ninguna modificación previa. El análisis encontró 46 violaciones, todas del tipo `[MissingJavadocMethod]`. El resto de reglas (nombres, imports, llaves, longitud de línea, etc.) no presentó ninguna violación, evidenciando que el equipo ya seguía buenas convenciones de estilo.

! BUILD FAILURE — 46 violaciones [MissingJavadocMethod]

3.1 Comando ejecutado y output

```
$ ./mvnw checkstyle:check

[INFO] --- checkstyle:3.3.1:check (default-cli) @ backend ---
[INFO] Comenzando auditoría...
[WARN] .../controller/ApuestaController.java:18:   Missing a Javadoc comment.
[MissingJavadocMethod]
[WARN] .../controller/ApuestaController.java:25:   Missing a Javadoc comment.
[MissingJavadocMethod]
[WARN] .../controller/EventoController.java:18:    Missing a Javadoc comment.
[MissingJavadocMethod]
[WARN] .../service/ApuestaService.java:22:         Missing a Javadoc comment.
[MissingJavadocMethod]
[WARN] ... (46 violaciones en total - ver tabla detallada abajo)
Auditoría concluida.
[INFO] You have 46 Checkstyle violations.
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
```

3.2 Distribución de violaciones por archivo

Paquete	Archivo	Violaciones	Tipo
repository	EventoRepository.java	1	MissingJavadocMethod
repository	HistorialCuotaRepository.java	1	MissingJavadocMethod
repository	ConfiguracionRiesgoRepository.java	1	MissingJavadocMethod
repository	MovimientoSaldoRepository.java	1	MissingJavadocMethod
repository	ApuestaRepository.java	1	MissingJavadocMethod
repository	UsuarioRepository.java	1	MissingJavadocMethod
repository	MercadoRepository.java	1	MissingJavadocMethod
controller	ApuestaController.java	4	MissingJavadocMethod
controller	EventoController.java	3	MissingJavadocMethod
controller	MercadoController.java	3	MissingJavadocMethod
controller	UsuarioController.java	3	MissingJavadocMethod
controller	ConfiguracionRiesgoController.java	2	MissingJavadocMethod
controller	MovimientoSaldoController.java	2	MissingJavadocMethod
service	ApuestaService.java	4	MissingJavadocMethod
service	EventoService.java	4	MissingJavadocMethod
service	MercadoService.java	4	MissingJavadocMethod

Paquete	Archivo	Violaciones	Tipo
service	UsuarioService.java	4	MissingJavadocMethod
service	MovimientoSaldoService.java	3	MissingJavadocMethod
service	ConfiguracionRiesgoService.java	2	MissingJavadocMethod
(raíz)	BackendApplication.java	1	MissingJavadocMethod
TOTAL	21 archivos	46	—

3.3 Análisis de las violaciones

La única regla violada fue MissingJavadocMethod. Esto indica que el equipo escribió el código siguiendo correctamente las convenciones de nombrado, estructura de llaves, orden de modificadores e imports, pero omitió la documentación formal de los contratos públicos. La ausencia de Javadoc es especialmente relevante en los servicios (17 violaciones) y controladores (17 violaciones), que constituyen la API pública de la aplicación.

4. Corrección aplicada

Se agregó documentación Javadoc descriptiva y específica a los 46 métodos públicos en los 21 archivos identificados, incluyendo @param, @return y @throws donde aplica.

4.1 Ejemplo de corrección aplicada

Antes (sin Javadoc):

```
// ❌ ANTES - sin documentación
public Optional<Evento> buscarPorId(Long id) {
    return eventoRepository.findById(id);
}
```

Después (con Javadoc descriptivo):

```
// ✅ DESPUÉS - documentado con @param y @return
/**
 * Busca y retorna un evento por su identificador único.
 *
 * @param id identificador del evento a buscar
 * @return Optional con el evento si existe, vacío si no se encuentra
 */
public Optional<Evento> buscarPorId(Long id) {
    return eventoRepository.findById(id);
}
```

4.2 Ejemplo con lógica de negocio — ApuestaService

```
/**
 * Registra una nueva apuesta en el sistema.
 * Congela la cuota actual de la opción seleccionada en el momento
 * de la apuesta (snapshot inmutable) e inicializa el estado en PENDIENTE.
 *
 * @param apuesta objeto con los datos de la apuesta a registrar
 * @return la apuesta persistida con cuotaCongelada y estado asignados
 * @throws IllegalArgumentException si la opción de apuesta no existe
 */
public Apuesta crear(Apuesta apuesta) { ... }
```

4.3 Archivos modificados

Paquete	Archivos	Métodos documentados
repository	7	7 (1 por archivo — finders derivados)
controller	6	17 (listar, buscar, crear + específicos)
service	6	21 (listar, buscar, crear + lógica de negocio)
(raíz)	1	1 (main de BackendApplication)
TOTAL	21	46 métodos con Javadoc agregado

5. Resultados — DESPUÉS de la corrección

✅ BUILD SUCCESS — 0 violaciones Checkstyle

5.1 Output del comando final

```
$ ./mvnw checkstyle:check

[INFO] --- checkstyle:3.3.1:check (default-cli) @ backend ---
[INFO] Comenzando auditoría...
```

```
Auditoría concluida.
[INFO] You have 0 Checkstyle violations.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

5.2 Comparativa antes / después

Métrica	ANTES	DESPUÉS
Total de violaciones	46	0 
Archivos con errores	21 / 21	0 / 21 
Tipo de violación	MissingJavadocMethod	—
Otras reglas violadas	0	0 
Resultado de compilación	BUILD FAILURE	BUILD SUCCESS 
Reporte HTML generado	Sí (46 items)	Sí (0 items) 

Nota: La verificación de compilación tras los cambios se realizó con `./mvnw compile` — BUILD SUCCESS confirmado.

6. Conclusiones

- **Calidad preexistente:** El código ya cumplía convenciones de nombrado, estructura de llaves, orden de modificadores y ausencia de wildcard imports — 0 violaciones en 13 de las 14 reglas.
- **Única área de mejora:** La ausencia de Javadoc en métodos públicos (46 violaciones) era el único problema, concentrado en servicios y controladores que forman la API pública del sistema.
- **Corrección aplicada:** Se documentaron los 46 métodos con Javadoc descriptivo incluyendo `@param`, `@return` y `@throws` donde corresponde, obteniendo 0 violaciones en la segunda ejecución.
- **Integración en el flujo:** La integración con Maven permite ejecutar `./mvnw checkstyle:check` desde cualquier entorno, facilitando su incorporación futura en un pipeline de CI/CD.